

Web Frontend Architecture (FROMO)

Overview

The FROMO web client is a single-page application (SPA) serving two distinct user surfaces from one codebase: a student-facing event-discovery interface and a partner/organiser management dashboard. It communicates with a FastAPI backend over REST and delegates identity/authentication to Supabase, while all business data (events, venues, bookings) flows exclusively through the backend API rather than being queried directly from the frontend.

Technology Stack

Layer	Technology	Version	Role
UI library	React	18.3	Component-based declarative rendering, hooks-based state
Build tool / dev server	Vite	5.4	ESM-native bundling, hot module replacement, production builds
Routing	React Router DOM	6.26	Client-side nested routing
Styling	Tailwind CSS	3.4	Utility-first CSS, class-based dark mode
CSS processing	PostCSS + Autoprefixer	—	Vendor prefixing, Tailwind pipeline
Mapping	Leaflet + react-leaflet	1.9 / 4.2	Interactive maps over CARTO basemap tiles (OpenStreetMap data)
Auth & session	Supabase JS client	2.45	Email/password auth, session/JWT issuance
HTTP client	Axios	1.7	REST communication with the FastAPI backend
Geocoding / routing	OpenRouteService (ORS) REST API	—	Address→coordinate geocoding, wheelchair-accessible walking directions
Event imagery	Ticketmaster Discovery API	—	Fallback event images when the backend has none

Layer	Technology	Version	Role
Testing	Vitest + React Testing Library + user-event + jsdom	2.1 / 16.3 / 14.6	Component/unit testing, coverage via @vitest/coverage-v8
CI/CD	GitHub Actions	—	Automated install → test → build pipeline on push/PR to main

Layered Architecture

The codebase separates concerns into four layers:

1. **Presentation layer** — `src/pages/` and `src/components/`. Pages are route-level screens (Home, Login, EventDetail, Bookings, and a `pages/partner/` subtree for the organiser dashboard); components (EventCard, EventMap, Header) are reusable, stateless-where-possible building blocks.
2. **Cross-cutting state layer** — `src/context/`. Rather than a dedicated state-management library (Redux, Zustand), the app uses React's Context API with two providers composed at the root (`main.jsx`): AuthProvider (Supabase session, user profile, sign-in/up/out) and ThemeProvider (dark/light mode, persisted to localStorage, applied via a data-theme DOM attribute that Tailwind's `darkMode: "class"` and CSS custom properties key off).
3. **Service/data layer** — `src/lib/`. Centralizes all I/O behind small exported functions: `api.js` wraps a shared Axios instance (`lib/api.js`) whose request interceptor reads the current Supabase session and attaches its JWT as a Bearer token to every outgoing request — giving the backend stateless authenticated calls without per-call token handling. `directions.js` wraps the OpenRouteService API for geocoding and routing; `ticketmaster.js` fetches fallback event images; `format.js` holds pure presentation-formatting helpers (currency, time, distance), kept separate from components specifically to keep them unit-testable in isolation.
4. **Backend integration** — a FastAPI REST API (separate service, Python) backed by PostgreSQL via Supabase. Supabase is used by the frontend purely as an identity provider (auth + JWT issuance); no business data is queried directly from Supabase client-side — every domain operation (events, venues, bookings, listings) goes through the FastAPI layer, giving a clean separation between authentication and domain logic.

Routing Structure

React Router v6 defines two route trees under one `<Routes>` in `App.jsx`:

- **Student/public**

tree: /, /login, /signup, /settings, /events/:id, /bookings, /checkout/success, /checkout/cancel.

- **Partner tree:** nested under /partner, wrapped by a PartnerLayout route element that supplies its own persistent shell (distinct from the student Header) with child routes for listings (index), analytics, settings, and bookings.

Authentication gating is handled at the component level rather than via a route-guard higher-order component: for example, clicking an EventCard checks the current user from AuthContext and either navigates to the event detail route or redirects to /login.

Testing & CI

Tests are colocated in __tests__/ folders next to the code they cover. Context dependencies (AuthContext) and router hooks (useNavigate) are isolated via vi.mock, so component tests exercise UI logic without a real backend or auth session. As of the current branch, the suite covers 7 test files / 60 tests (components, pages, and the lib service layer). A GitHub Actions workflow (ci-web.yml) runs on every push/PR touching web/**: npm ci → npm test (Vitest) → npm run build (Vite production build) on Node 20 LTS.